

Loyola Marymount University

CMSI 3300 Artificial Intelligence

Fall 2024

Professor Lanyu Shang

Announcements

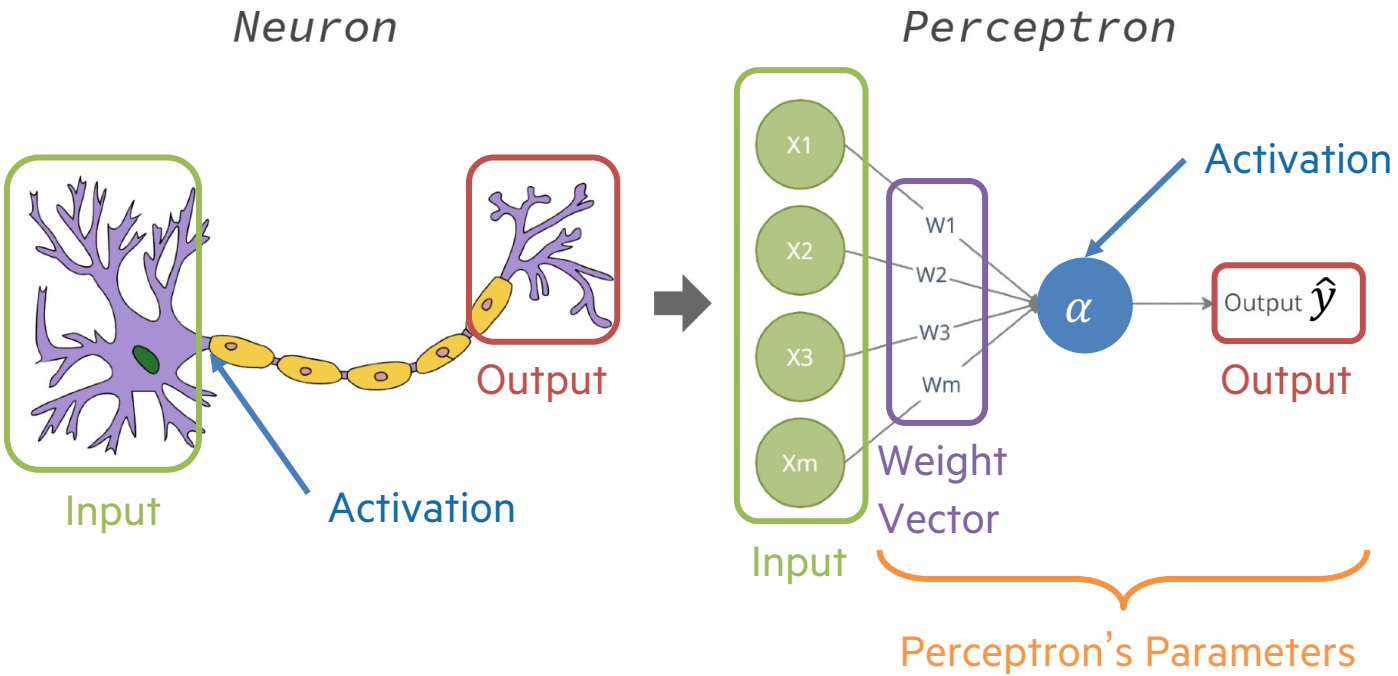
- The grades of Homework 2 are available on Brightspace.
- 11/15 is the last day to withdraw or apply for credit/no credit grading
- Homework 3
  - Due this Friday, 11/15 @5pm

This Lecture

- Perceptron
  - Learning Perceptron
  - Perceptron Properties
  - Fixing Perceptron
- Logistic Regression

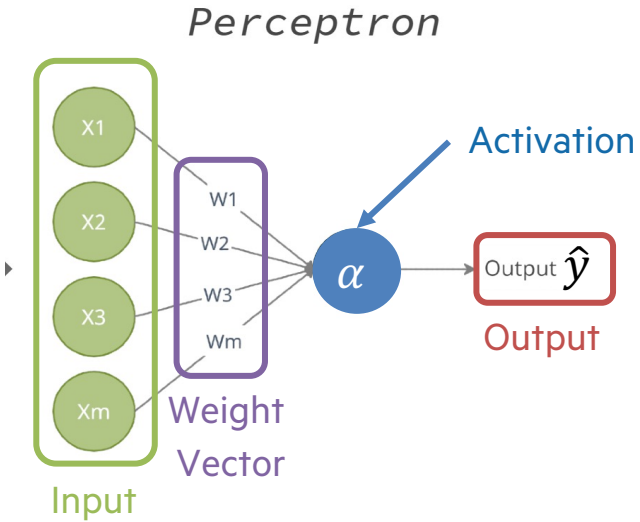
Last Lecture

- Perceptron!



Perceptron's Parameters

- Weight Vector $\boldsymbol{w}$
  - $\boldsymbol{w} =< w_1, w_2, \dots, w_n >$
- Activation α
  - $\alpha = \boldsymbol{w} \cdot \boldsymbol{x} = \sum_i w_i \cdot x_i$
- Decision Rule
  - Binary: $\hat{y} = (\text{activation}_{\boldsymbol{w}}(\boldsymbol{x}) > 0)?\, 0 : 1$
  - Multiclass: $\hat{y} = \text{argmax}_y \alpha_y = \text{argmax}\, \boldsymbol{x} \cdot \boldsymbol{w_y}$



Where do they come from?

Learning Perceptron

- Learning Weight Vector $\boldsymbol{w}$
  - Top-Down or Bottom-Up?
    - Bottom-Up: Learning from the training data
    - How?
  - Example: two features, two classes
    - Training data: $\boldsymbol{x} =< -1, 3 >$ and $y = 0$
    - $\boldsymbol{w} =< 1, 1 >$
    - $\alpha = 1 * -1 + 1 * 3 = 2$
    - $\hat{y} = 1 \neq y$, how to fix it?

Adjust $\boldsymbol{w}$, how?

α is too large

Linear Perceptron Update Algorithm

Initialize Weights

```
# Initialize all class' weight vectors to all 0s
w_y = <0, 0, ..., 0>
```

Predict w/ Current Weights

```
# Go through each example in training set
for each x in training_set:
    # Make prediction with current weights
    y_hat = argmax_y (w_y • x)
```

Update Weights

```
    # Let's call y* the "right" answer from training
    # If our prediction y_hat is right, yay! Do nothing... else:
    if y_hat != y*:
        # Reduce weights of false positive
        w_{y_hat} = w_{y_hat} - x
        # Increase weights of correct answer
        w_y* = w_y* + x
```

Example

- 3 features, 3 classes {0,1,2}
  - $\boldsymbol{w_0} = < 1,0,2 >, \boldsymbol{w_1} = < -1,2,0 >, \boldsymbol{w_2} = < 0, -2,2 >$
  - Sample: $\boldsymbol{x} = < 1,2,3 >,\ y = 1$
  - Compute the update to the weights.
- Step 1: Compute the activation for each class
  - $\alpha_0 = 1 + 0 + 6 = 7$
  - $\alpha_1 = -1 + 4 + 0 = 3$
  - $\alpha_2 = 0 - 4 + 6 = 2$

Example

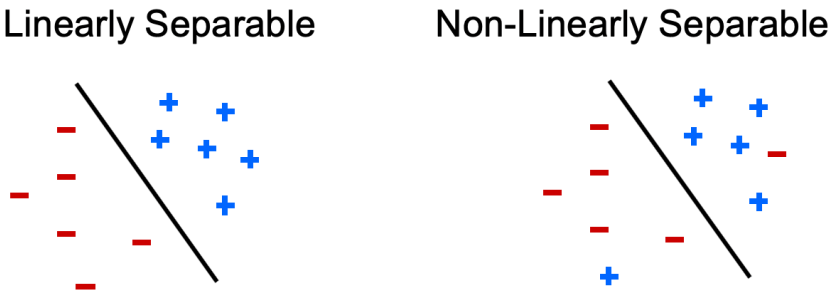
- Step 2: Select class by decision rule
  - $\hat{y} = \text{argmax}_{\mathbf{y}} \alpha_y = 0$
- Step 3: Update weights if wrong or keep them if right.
  - Which weight(s) need to be updated?
    - Penalize
      - $\boldsymbol{w_0} = \boldsymbol{w_0} - \boldsymbol{x} = < 1,0,2 > - < 1,2,3 > = < 0, -2, -1 >$
    - Accentuate
      - $\boldsymbol{w_1} = \boldsymbol{w_1} + \boldsymbol{x} = < -1,2,0 > + < 1,2,3 > = < 0,4,3 >$

Linear Perceptron Update

- Visualization: https://vinizinho.net/projects/perceptron-viz/

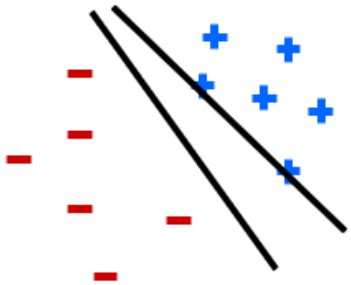
Perceptron Properties

- Separability
  - Decision boundaries can partition the labeled data
  - If a training set is **separable**, the perceptron update rule converges to one such choice for optimal weights.
  - Linearly Separable



Perceptron Problems

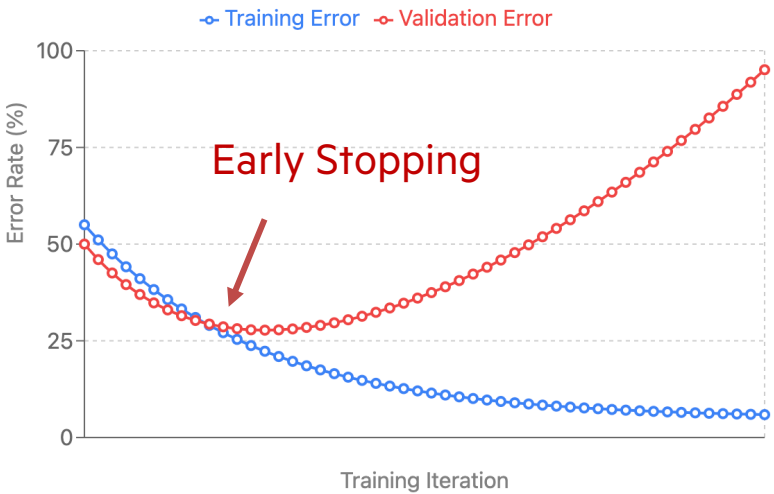
- Convergence Issue
  - If a training set is **not** linearly separable, weights may thrash back and forth => prevent convergence
- Generalization Issue
  - If a training set **is** linearly separable, some decision boundaries will **better** generalize than others.
  - The perceptron isn't guaranteed to find the best.

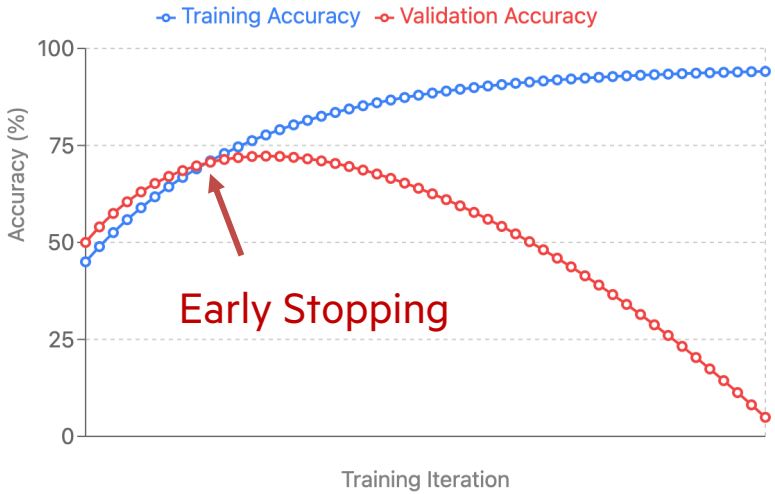


Perceptron Problems

- Overfitting

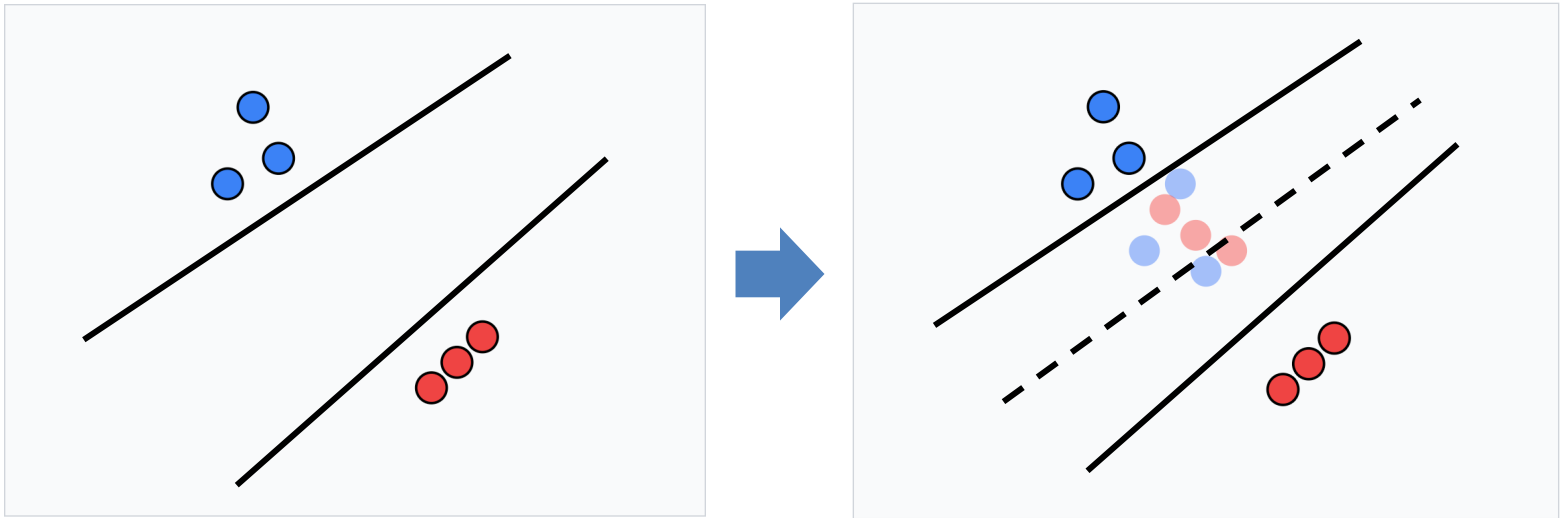
How to fix it?





Fixing Perceptron

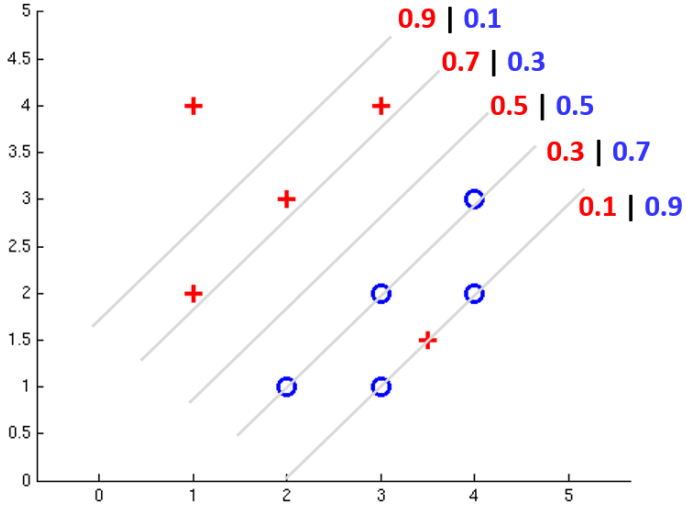
- Generalization Issue



Solid: Training Set
Transparent: Test Set

Fixing Perceptron

- "Soft" Decision Boundary
  - Each decision boundary now encodes a *likelihood* of samples on each side belonging to one class over the other



Logistic Regression

- Linear perceptron but with different activation and decision rule
  - Activation: sigmoid function $\phi(z) = \sigma(z)$
    - Scalar z => prob. Proportionate to magnitude

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

